Type-safe tensors for humans

Jan Ypma

October 27, 2026

Outline

1. Type-safe tensors for humans
   - Getting started
   - Introduction to tensors
   - Introduction to machine learning
   - What about human learning?
2. Type systems
   - The need for types
   - Generics
   - Tuples
   - Implicit arguments
3. Types for tensors
   - Pytorch
   - Use on the JVM
   - Type definitions
4. Selected tensor operations
   - Element-wise operations
   - Scalar functions (sin, relu, etc.)
   - Comparing tensors
5. Dimension tricks
   - Selecting dimensions
   - Comparing dimensions statically
   - Dividing and multiplying dimensions
   - Safely defining givens for dynamic dimensions
   - Dynamic batch dimensions
6. Practical examples
   - Example: Modeling cross entropy loss
   - Example: Applying a head of self-attention

About me

- Jan Ypma
  - Independent software architect
  - Java, Scala, Groovy, C++, Rust, Lisp
  - Contributor to various open source projects
  - Functional programming and distributed systems
  - Agile coach and fan of Domain-Driven Design
  - jan@ypmania.net, https://linkedin.com/in/jypma
- This talk: https://github.com/jypma/s3torch/tree/master/presentation

Tensor

Wikipedia:
An algebraic object that describes a multilinear relationship between sets of algebraic objects associated with a vector space.
For our cause:
A multidimensional linear data structure of numbers

Dimensionality examples

- A matrix is a tensor with 2 dimensions

```
((
  ((1, 2, 3)),
  ((0, 2, 0))
))
```

- A vector is a tensor with 1 dimension

```
((1, 2, 3))
```

- A tensor with 0 dimensions is a scalar
  - just a number

Machine learning model

```
(limited number of inputs) --> (very many mathematical operations,
↪  steered by weights) --> (limited number of outputs)
```

Having the *machine* do the *learning*:

- Take a large number of examples of correct inputs and outputs, one at a time
- See how the current weights perform on that input
- Poke at the weights to make them a little better
- Rinse and repeat

What direction do we poke at?

Gradient descent

```
(limited number of inputs) --> (very many mathematical operations,
↪  steered by weights) --> (limited number of outputs) --> (loss)
```

- Let's limit the allowed operations to only *differentiable* ones
- We combine the actual outputs with the expected outputs into a *loss* figure
- Calculate the partial derivative of each weight towards that loss
- Poke the weights such, that the loss (hopefully) decreases

Optimizing performance

Avenues for parallelization:

- Modern models have a lot of weights
  - They tend to depend on each other
- We do have a lot of training examples to go through
  - Those are independent!

Solution: process in *batches*

Consider this python example

```
class PositionalEncoding(nn.Module):

    def __init__(self, d_model: int, seq_len: int, dropout: float) ->
    ↪  None:
        super().__init__()
        self.d_model = d_model
        self.seq_len = seq_len
        self.dropout = nn.Dropout(dropout)
        # Create a matrix of shape (seq_len, d_model)
        pe = torch.zeros(seq_len, d_model)
        # Create a vector of shape (seq_len)
        position = torch.arange(0, seq_len,
        ↪  dtype=torch.float).unsqueeze(1) # (seq_len, 1)
        # Create a vector of shape (d_model)
        div_term = torch.exp(torch.arange(0, d_model, 2).float() *
        ↪  (-math.log(10000.0) / d_model)) # (d_model / 2)
        # Apply sine to even indices
        pe[:, 0::2] = torch.sin(position * div_term) # sin(position *
        ↪  (10000 ** (2i / d_model))
        # Apply cosine to odd indices
        pe[:, 1::2] = torch.cos(position * div_term) # cos(position *
        ↪  (10000 ** (2i / d_model))
        # Add a batch dimension to the positional encoding
        pe = pe.unsqueeze(0) # (1, seq_len, d_model, 1)
        # Register the positional encoding as a buffer
        self.register_buffer('pe', pe)
```

Can you follow along what the tensor dimensions are?

Rationale

- Why do programming languages have, or introduce, types?

Rationale (2)

- Why do programming languages have, or introduce, types?

```
def average(number1, number2, number3) = {
    (number1 + number2 + number3) / 3
}
```

Rationale (3)

- Why do programming languages have, or introduce, types?

```
def average(number1, number2, number3) = {
    (number1 + number2 + number3) / 3
}

average(1, 2, 3)
average("1", "2", "3")
average(42, Clock.now(), File.read("/cat.jpg"))
```

Early detection

Make invalid states unrepresentable

```
def average(number1: Int, number2: Int, number3: Int): Int = {
    (number1 + number2 + number3) / 3
}
```

- Not possible to write code that puts something else in there
- Early detection of potential issues
  - Cost of fixing defects goes up exponentially with time (source: testomat/IBM and source: forbes)

The case for generics

- We want to store lists of things, so let's imagine a type `List`

```
def getFirstElement(items: List): ???
```

- What should the return type be?

Lists of things

- Let's give list a *type parameter*, so we can define what's in the list

```
val numbers: List[Int] = List(1, 2, 3)
val cats: List[Cat] = List(Cat.load("/kitten.jpg"))

def getFirstElement(items: List[Int]): Int
def getFirstElement(items: List[Cat]): Cat
```

Lists of anything

- Our function doesn't care about what's in the list
- But still wants to return the right type

```
def getFirstElement[T](items: List[T]): T
```

Combining types

Consider a generic "map" data structure

```
trait Map[K, V]
```

What is a good type for one entry in the map?

Tuple

A *tuple* is an ordered, fixed-length sequence of values of known (but potentially distinct) types.

```
case class Tuple2[A, B](a: A, b: B)

case class Tuple3[A, B, C](a: A, b: B, c: C)

// etc.
```

Type classes

We want to write a sort algorithm:

```
def sorted[T](list: List[T]): List[T]
```

Now we need a way to compare values of `T`.

- Interface: not an option, we don't want to limit what `T` we can put in
- Alternative: Helper object ("type class")

```
trait Comparator[T] {
  def compare(a: T, b: T): Int
}
def sorted[T](list: List[T], comparator: Comparator[T]): List[T]
```

But passing this around every time is cumbersome.

Given and using

We can have Scala pass the argument automatically.

```
def sorted[T](list: List[T])(using comparator: Comparator[T]): List[T]
```

Then, make sure a "given" value is in scope:

```
given comparator: Comparator[T] = ???

val myList: List[T] = ???
val result = sorted(myList)
```

Introduction

- Extremely popular due to its easy of use, and (mostly) single API for both CPU and GPU inference
- Python api (`pytorch`) on top of a C++ backend (`libtorch`), with various hardware-dependent extensions
- API for wide variety of tensor operations
  - General math, matrix operations, cross entropy
  - Composing frequently used building blocks into modules
  - Everything is tracked for gradient descent back-propagation
- Tensors
  - are *mutable*
  - have a fixed *size* with a number of *dimensions*
  - have a fixed *data type* (64-bit integer, 32-bit float, etc.)
  - are stored on a specific *device* (e.g. CPU, GPU number 2, etc.)

Runtime restrictions

Not all pytorch operations accept any input (just like python):

- Data type requirements
  - nn.Embedding (see "Shape"): Input has to be `IntTensor` or `LongTensor`, output can be anything
- Dimension requirements
  - nn.functional.cross$_{\text{entropy}}$ (see "Shape"): Very specific input shapes accepted. Output depends on what input shape was passed.
- Data value requirements
  - nn.Embedding: Input values have to "fit" the size of the embedding (lookup table)

"Bytedeco" project

- Wrappers by Samuel Audet to provide automatic, convenient JNI wrappers for C++ libraries
- Access to memory management, pointers, even streams
- Scala project from which this work was inspired: storch
  - Scala wrapper around `libtorch`
- Extended into `https://github.com/jypma/s3torch` (originally a fork of `storch`)

Data type

Let's model this as an abstract class.

```
abstract class DType(val native: torch.ScalarType)

object DType {
  abstract class BFloat16 extends DType(torch.ScalarType.BFloat16)
  val bfloat16 = new BFloat16 {}
  abstract class Bool extends DType(torch.ScalarType.Bool)
  val bool = new Bool {}
  abstract class Int8 extends DType(torch.ScalarType.Char)
  val int8 = new Int8 {}
  abstract class Int16 extends DType(torch.ScalarType.Short)
  val int16 = new Int16 {}
  // [...]
```

An let's have `Tensor` be generic in its data type:

```
class Tensor[T <: DType /* , ... */]
```

Device

Tensors can generally only interact with tensors on the same device. So, let's make other interactions *unrepresentable*.
We'll define an abstract class representing a `libtorch` device.

```
abstract class DeviceType(val native: torch.DeviceType)

abstract class Device(val deviceType: DeviceType, val index: Byte = -1)

case object CPU extends Device(DeviceType.CPU)
case object CUDA extends Device(DeviceType.CUDA)
```

And let's have `Tensor` be generic in the device type:

```
class Tensor[T <: DType, D <: Device /* , ... */]
```

Shape

Any suggestions on how to represent the tensor shape (dimensions)?

Dimension

Let's first look at what a *dimension* actually is, from a program's perspective.

```
trait Dim extends Ordered[Dim] {
  def size: Long
}
```

Dimension (static)

Some dimensions are known at compile time.

```
abstract class Static[S <: Long](using ValueOf[S]) extends Dim {
  type Size = S
  def size = valueOf[S]
}
```

This allows us to name static dimensions:

```
case object BatchSize extends Static[32L]
case object NumberOfHeads extends Static[4L]
```

Dimension (dynamic)

Some dimensions depend on data we read in at run-time.

```
class Dynamic(_size: Long) extends Dim {
  override def size = _size
}
```

We can still give them a name:

```
case object VocabularySize extends Dynamic(loadedDictionary.size)
```

The compiler knows nothing about these... initially.

Revisiting shape

- A tensor's shape might have any number of dimensions
- We want to encode each dimension as a type
- Let's use Scala 3's `Tuple` for this

```
case object BatchSize extends Static[32L]
case object NumberOfHeads extends Static[4L]

type ShapeOfMatrix = (BatchSize, NumberOfHeads) // Tuple2[BatchSize,
↪  NumberOfHeads]
type ShapeOfVector = Tuple1[BatchSize]
type Scalar = EmptyTuple
```

Conveniently, all tuple types in Scala 3 are `<: Tuple`.
Hence, our final tensor type:

```
class Tensor[S <: Tuple, T <: DType, D <: Device](val native:
↪  pytorch.Tensor)
```

Considerations

Let's investigate element-wise operations

- +, -, *, /, ...
- Pairwise boolean comparison (<, >, equal, ...)

For what shapes should we support this?

- Same shape on both sides
- Different shapes: `pytorch` broadcasting rules

And what data types?

- Different data types: `pytorch` promotion rules

Promotion example

```
it("can plus an int vector with a float vector") {
  val v1 = Tensor((1, 2, 3))
  val v2 = Tensor((1.0, 2.0, 3.0))
  val res = v1 + v2
  val resType: Tensor[Tuple1[Static[3L]], Float64, CPU.type] = res
  assert(res.size == Seq(3L))
  assert(res.value.toSeq == Seq(2.0, 4.0, 6.0))
}
```

Broadcasting example

```
it("can plus a matrix with a vector") {
  val m = Tensor((
    ((1, 2, 3)),
    ((0, 4, 0))
  ))
  val v = Tensor((1, 2, 3))
  val res = m + v
  val resType: Tensor[(Static[2L], Static[3L]), Int32, CPU.type] = res
  assert(res.size == Seq(2L, 3L))
  assert(res.value.toSeq == Seq(
    Seq(2, 4, 6),
    Seq(1, 6, 3)
  ))

  // Test symmetry
  assert((v + m).value.toSeq == Seq(
    Seq(2, 4, 6),
    Seq(1, 6, 3)
  ))
}
```

Boolean example

```
it("can compare two tensors of different type") {
  val a = Tensor((1, 2, 3))
  val b = Tensor((1.0, 42.0, 3.0))
  val res = a #== b // Element-wise equality
  val resType: Tensor[Tuple1[Static[3L]], Bool, CPU.type] = res
  assert(res.size == Seq(3L))
  assert(res.value.toSeq == Seq(true, false, true))
}
```

Picking syntax

Same operation on each element

- output has same shape (and usually same type) as input

Alternative A

Member function

```
val result = myTensor.sin
val result = myTensor.relu // recified linear unit
```

- Idiomatic object orientation
- Nice to "chain" calls

Alternative B

Function in global scope

```
val result = sin(myTensor)
val result = relu(myTensor)
```

- Math notation
- Pytorch choice
- `s3torch` for familiarity

. . . but "chained" calls will be back!

Defining equality

When are two tensors "equal"?

- Same dimensions? Well, yeah
- Same value in each position? Yes
- Same device? [...]

```
override def equals(that: Any): Boolean = that match {
  // Check dtype and device, since pytorch will just crash if they don't
  ↪  match.
  case other: Tensor[?, ?, ?] if dtype == other.dtype && device ==
  ↪  other.device =>
    native.equal(other.native)
  case _ =>
    false
}
```

The need to select dimensions

- Find the (*maximum*, *mininum*, *average*, ...) across one dimension

```
val t = Tensor((
  ((1.0, 5.0, 3.0)),
  ((4.0, 2.0, 6.0))
)).shaped[(Row, Column)]

val r = t.maxBy(dim[Row])
```

- What would you expect the type of `r` to be?

Source: TensorSpec.scala

Compile-time safety for picking dimensions

All `Tensor` methods that require referring to a dimension can do this in several ways:

```
case class Row extends Dim.Static[2L]
case class Column extends Dim.Static[4L]
val t = Tensor.zeros(Row, Column)
```

- By dimension value: `t.maxBy(Row)`
- By dimension type (if a value isn't available): `t.maxBy(dim[Row])`
- By first or last: `t.maxBy(Select.First)` or `t.maxBy(Select.Last)`
- By a specific index (must be a compile-time constant) `t.maxBy(Select.Idx(0))`

Of course, all of the above are compile-time verified.

Case: pad to given size

```
/** Returns a new tensor with the last dimension padded to [dim]. [dim]
↪  must be at least as large as the current last dimension. */
def padTo[D <: Dim, V](dim: D, value: V)(using /* ... */) = ???
```

We want compile-time proof that

- the *last* dimension of `S` (the shape of our tensor)
- is LESS or equal than `D`

The "last" dimension

Let's define that as a given :

```
trait LastDim[S <: Shape] {
  type D <: Dim
}
object LastDim {
  given [A <: Dim]: LastDim[Tuple1[A]] with {
    type D = A
  }
  given [S <: Tuple, A <: Dim](using l: LastDim[S]): LastDim[A *: S]
  ↪  with {
    type D = l.D
  }
}
```

Knowledge of `A < B`

Let's define a `given` for that as well.

```
/** Proof that A is less than or equal to B */
trait LessOrEqual[A <: Dim, B <: Dim] {}
object LessOrEqual {
  // definitions here for statically-known dimension types
}
/** Proof that A is less than or equal to B */
infix type |<=[A <: Dim, B <: Dim] = LessOrEqual[A, B]
```

Now, we can revisit padTo:

```
def padTo[D <: Dim, V](dim: D, value: V)(using last: LastDim[S], ev:
↪  last.D |<= D) =
```

The need for `split`

Let's define the ability to "split up" one dimension into two new ones.

- We often want to do this to calculate parts separately, and later "merge" them again

Let's say, we have

```
case object DimA extends Static[6L]
case object DimB extends Static[3L]

val matrix = Tensor.zeros(DimA, DimB)
```

and we do something like

```
val res = matrix.view.split(DimA).into(Dim.Static(2L))
```

What do we expect the dimensions of `res` to be?
What would be a reasonable Scala type for that?
When should it be allowed to "split" a dimension `A` into `B` parts?

The need for `split` (cont.)

We want to remember that it was `DimA` that we divided by 2:

```
case object DimA extends Static[6L]
case object DimB extends Static[3L]

val matrix = Tensor.zeros(DimA, DimB)

val res = matrix.view.split(DimA).into(Dim.Static(2L))

val resType: Tensor[(Static[2L], DimA.type / Dim.Static[2L],
↪  DimB.type), Float32, CPU.type] = res
assert(res.size == Seq(2L, 3L, 3L))
```

We can do this similar to how we modeled the "less than" proof:

```
/** Proof that D is divisible by L */
trait DivisibleBy[+D <: Dim, +L <: Dim] {}
infix type |/[+D <: Dim, +L <: Dim] = DivisibleBy[D, L]

/** A Dim that is the result of dividing two other Dims */
trait DividedDim[D <: Dim, L <: Dim] extends Dim {}
infix type /[D <: Dim, L <: Dim] = DividedDim[D, L]
```

- See the python way in "nanoGTP", model.py

Merging dimensions

We want to remove a dimension from a tensor, merging it into the one before it:

```
val matrix = Tensor.zeros(DimA, DimB)

val res = matrix.view.merge(DimB)
```

What do we expect the dimensions of `res` to be?
What would be a reasonable Scala type for that?
When should it be allowed to "merge" a dimension `A` into the one before it?

Merging dimensions (cont.)

We want to remember which dimensions we've merged:

```
/** A Dim that is the result of multiplying two other Dims */
trait ProductDim[A <: Dim, B <: Dim] extends Dim
infix type *[A <: Dim, B <: Dim] = ProductDim[A, B]
```

We can now use a bit of given magic to do "the right thing":

```
val matrix = Tensor.zeros(DimA, DimB)

val res = matrix.view.split(DimA).into(Dim.Static(2L))
val resType: Tensor[(Static[2L], DimA.type / Dim.Static[2L],
↪  DimB.type), Float32, CPU.type] = res

val un = res.view.merge[DimA.type / Dim.Static[2L]]
val unType: Tensor[(DimA.type, DimB.type), Float32, CPU.type] = un
```

Declaring givens

Say that we have two dynamic dimensions:

```
def myMethod[A <: Dim, B <: Dim](a: A, b: B) = {
  val vector: Tensor[A] = Tensor.zeros(a)
  val bigger = vector.padTo(B, 0)
}
```

We now have to "prove" to the compiler that `A <= B`. We could do:

```
if (a.size <= b.size) {
  given A |<= B with {}
  val bigger = vector.padTo(B, 0)
}
```

and that will work, but seems double-up.

Automatic givens

Instead we can write:

```
for {
  ev <- a |<= b
} {
  import ev.given
  val bigger = vector.padTo(B, 0)
}
```

or even a loop, using a simple helper (extending an example up to a `MaxBlockSize`):

```
whileDefined(gen.sizeOf(ExampleSize(_)) |<= MaxBlockSize) { ev =>
  import ev.given
  gen = model.extend(gen)
}
```

Introduction

Let's say we have written

```
def myAwesomeCalculation[D <: Device](in: Tensor[(Row, Col), Float32,
↪  D]): Tensor[EmptyTuple, Float32, D] = ???
```

We've heard that allowing this to be calculated in batches is good for performance. How does our signature change?

Add explicit batch dimension

Let's add an extra dimension to represent our "batch"

```
def myAwesomeCalculation[D <: Device, Batch <: Dim](in: Tensor[(Batch,
↪  Row, Col), Float32, D]): Tensor[EmptyTuple, Float32, D] = ???
```

We can now submit calculations with varying batch sizes.
But what about our original example without a batch at all?

Making batch dimension(s) optional

Many operations in `libtorch` accept (an) optional batch dimension(s).

- Not all of them
- Not always documented
- Sometimes one batch dimension, sometimes several

Let's define a trait that covers:

- Some unknown batch dimensions `B <: Tuple`, with known dimensionality
- Some unknown non-batch dimensions `T <: Tuple`, with known dimensionality

```
/** Proof that shape [S] is in fact the concatenation of (potentially
↪  call-site unknown) batch dimensions [B]
    with actual, known dimensions [T] */
trait Batched[B <: Tuple, T <: Tuple, S <: Tuple](using
↪  b:ValueOf[Size[B]], t:ValueOf[Size[T]])
```

Example: `Linear`

A *linear* layer in a neural network:

- Accepts as input tensors that have `In` as a final dimension
- Has an internal matrix of weights, that transforms that `In` dimension into an `Out` dimension

In other words:

- Input must consist of any batch dimensions, and then one dimension `In`.
- Output will be those batch dimensions, and then `Out`

```
def apply[S <: Tuple, B <: Tuple, T <: DType, Idx <: Int](in: Tensor[S,
↪  T, D])(using Batched[B, Tuple1[In], S]): in.Shaped[B :* Out]
```

Actual definition: Linear.scala

Simplified description

- We've calculated `B` examples with our model.
- Each example holds a vector of probabilities for what decision the model deems most likely (next token, dog or brownie, ...).
- We also have `B` *expected* outcomes (from our training data)

In pytorch, this is torch.nn.CrossEntropyLoss

Scala representation

Let's model a simplified version in Scala.

```
def crossEntrypyLoss[
  SI <: Tuple, // Input shape
  ST <: Tuple, // Target shape
  D <: Device,
  TI <: DType, // Input DType
  TT <: DType.Int64 // Target DType. Undocumented, but pytorch/libtorch
  ↪  only support Int64 here.
] (
  input: Tensor[SI, TI, D], target: Tensor[ST, TT, D]
) (using
  ValidShape[SI, ST]
): input.Shaped[Scalar]
```

Which shapes do we accept?

```
trait ValidShape[I <: Tuple, T <: Tuple]
object ValidShape {
  given s1[C <: Dim]: ValidShape[C *: EmptyTuple, EmptyTuple] with {}
  given s2[N <: Dim, C <: Dim, Tail <: Tuple]: ValidShape[(N *: C *:
  ↪  Tail), N *: Tail] with {}
}
```

See it in action: CrossEntropySpec

Introduction

(Very) simplistic sketch of a *transformer* model:

```
(input tokens) --> (sequence of DModel-dimensional elements) -->
↪  (transformations) --> (output tokens)
```

- Input tokens, and resulting sequences, typically come from code or text
- Each *transformation* converts one sequence of `DModel`-dimensional elements into a new one
- Final step down-converts the `DModel` dimensions back into tokens.
- Let's look at a single one of these transformations (called a *head*)
  - Internally, the *head* has a matrix that of weights
    - Correlates elements on different positions to each other

Initial definition

```
type DModel <: Dim // or could be a Dim.Static

def performCalculation[L <: Dim](x: Tensor[(L, DModel), Float32, D]):
↪  Tensor[(L, DModel), Float32, D]
```

Adding a batch dimension

We want to be able to calculate many examples simultaneously.

```
def apply[B <: Dim, L <: Dim](x: Tensor[(B, L, DModel), Float32, D]):
↪  Tensor[(B, L, DModel), Float32, D]
```

Adding dimension limitations

Adding a limitation on `L` less than the block size (because we use `Take`)

- The *head*'s internal matrix only has weights for sequence elements up to `MaxBlockSize` length.

```
def apply[B <: Dim, L <: Dim](x: Tensor[(B, L, DModel), Float32,
↪  D])(using L |<= MaxBlockSize): Tensor[(B, L, DModel), Float32, D]
```

Parameterizing the batch dimension

```
def apply[S <: Tuple, B <: Tuple, L <: Dim](x: Tensor[S, Float32,
↪  D])(using b: Batched[B, (L, DModel), S])(using L |<= MaxBlockSize):
↪  Tensor[B ++ (L, DModel), Float32, D]
```

Concatenate along dimension

Let's say we have a known number of tensors (`Num`), that we want to concatenate along a dimension (`Row`).

- What's the size of that dimension in the resulting tensor?
- How can we capture the concept of "`Num` tensors" at compile time?

Concatenate along dimension (TensorValue)

We make a simple wrapper around Seq.

```
/** A sequence of values with compile-time known shape S. */
class TensorValue[S <: Tuple, +T](private val contents: Seq[T]) extends
↪  Iterable[T] {
  def map[U: ClassTag](fn: T => U): TensorValue[S, U]  = new
  ↪  TensorValue(contents.map(fn))
}

object TensorValue {
  def arangeOf[D <: Dim](dim: D): TensorValue[Tuple1[D], Long] = new
  ↪  TensorValue((0L.until(dim.size)))
}
```

Concatenate along dimension (cat)

Now ,we can use `TensorValue` to represent a sequence of same-shaped Tensors of known size.

```
def cat[D <: Dim, S <: Tuple, T <: DType, Dv <: Device, A](
  tensors: TensorValue[Tuple1[D], Tensor[S, T, Dv]],
  alongDim: A
)(using
  ...
):
```

The full signature `using` clause calculates the resulting shape of `A * D`:

```
case object CatDim extends Dim.Static[3L]
val ts = TensorValue.arangeOf(CatDim).map(_ => Tensor.zeros(4L, 5L, 6L))
val tsType: TensorValue[Tuple1[CatDim.type], Tensor[(Static[4L],
↪  Static[5L], Static[6L]), Float32, CPU.type]] = ts

val r = Tensor.cat(ts, Select.First)
val rType: Tensor[(CatDim.type * Static[4L], Static[5L], Static[6L]),
↪  Float32, CPU.type] = r
assert(r.size == Seq(CatDim.size * 4L, 5L, 6L))
```

Evaluate a function long a dimension

Imagine we have a tensor with some values:

```
case object DimA extends Dim.Static[3L]
val t = Tensor.zeros(DimA)
```

Now, we want to execute a function on each element in `t`, calculating a new `Tensor`

- What should the resulting shape be?

```
val m = t.mapStack(v => Tensor.zeros(DimB))
val mType: Tensor[(DimA.type, DimB.type), Float32, CPU.type] = m
```

Evaluate a function long a dimension (cont.)

Imagine we have a tensor with some values:

```
case object DimA extends Dim.Static[3L]
val t = Tensor.zeros(DimA)
```

Now, we want to execute a function on each element in `t`, calculating a new `Tensor`, and concatenate the results

- What should the resulting shape be?

```
case object DimB extends Dim.Static[2L]
case object DimC extends Dim.Static[4L]
val r = m.mapCat(v => Tensor.zeros(DimB, DimC), Last)
val rType: Tensor[(DimB.type, DimA.type * DimC.type), Float32,
↪  CPU.type] = r
```

Chained invocations

- No need for `module.Sequential`. Actual chained invocations retain type

```
import Tensor.relu
val linear = addModule("linear", Linear(dModel, dModel))

def apply(Tensor[/* ... */] in) = relu(linear(in))
```

However, `Tensor` defines `~>` which allows us to write this as

```
in ~> linear.apply ~> relu
```

Here's a more complete module:

```
class FeedForward(dropoutProb: Double) extends Module {
  val l1 = addModule("l1", Linear(dModel, dff))
  val dropout = addModule("dropout", Dropout(dropoutProb))
  val l2 = addModule("l2", Linear(dff, dModel))

  def apply[B <: Shape, SeqLen <: Dim, S <: Shape](in: Tn[S])(using
  ↪  b:Batched1[B, DModel, S]): Tn[B :* DModel] = {
    import b.given
    in ~> l1.apply ~> relu ~> dropout.apply ~> l2.apply
  }
}
```

- Reference counting

References to (CPU) objects are *immediately* garbage collected, once no longer referenced:

```
import torch

for i in range(10):
    x = torch.randn(10000, 10000).cuda()
    print(torch.cuda.max_memory_allocated())
```

gives:

```
400556032
801112064
801112064
801112064
[...]
```

- Cuda will re-use its tensor memory once the CPU object becomes GC'ed.

- Garbage collector
  - JVM has a complicated process to decide when memory is released
  - Usually not immediately when no longer referenced
  - Usually, a bunch at once

PointerScope

- javacpp has a generic solution: `PointerScope`
  - Thread-l0cal storage to which all allocated pointers are added
  - All pointers can be released as a single scope
  - Ideal to wrap in a Scala block

```
import net.ypmania.s3torch.HeapExternal.scoped

val myResult = scoped {
  val intermediate = model.performCalculation(input)
  infer(intermediate)
}
```

- Everything allocated inside `scoped` is immediately freed
  - *except* the result

So, what have we learned?

- Pytorch documentation vs. reality
- Tracing dimensions is fun!
  - Improves clarity of calculations
- Subtle bugs now fixable
  - Two dimensions equal does not mean it's the same dimension
- Scala limitations and challenges
  - No way to express "a tuple where all elements `<: X`"
  - Match types vs. givens
  - Still the odd compiler bug when exploring complex type setups
  - How do we refrain from implementing a full CAS in Scala's type system?

Next steps

- Type (and range) safe indexing data types
- Better alignment with `pytorch`'s auto-selected data types
- Find out more

https://github.com/jypma/s3torch `jan@ypmania.net`